



SQL Project Data Analysis

PIZZA SALES

- ANALYZING TRENDS AND INSIGHTS
FOR IMPROVED DECISION-MAKING





INTRODUCTION

[About](#)

[Contact](#)

OBJECTIVE

ANALYZE PIZZA SALES DATA TO UNCOVER
TRENDS AND INSIGHTS.

Use SQL queries to extract actionable information.

Understanding sales patterns helps businesses make informed decisions.

Pizza sales analysis improve inventory management, marketing, and customer satisfaction.

SCOPE

Focused on sales, customer preferences, and revenue analysis.

TOOLS USED

MySQL Workbench is used For writing and executing SQL queries.





DATASET OVERVIEW

[About](#)[Contact](#)

THE DATASET COMPRISES FOUR FILES THAT PROVIDE A COMPREHENSIVE VIEW OF PIZZA SALES, CAPTURING ALL ESSENTIAL DETAILS FOR ANALYSIS..

Orders: file

Contains information about customer orders.

Key attributes: Order ID, Customer ID, Order Date.

Order Details:

Provides details of each order, including the pizzas ordered and quantities.

Key attributes: Order ID, Pizza ID, Quantity.

Pizza Types:

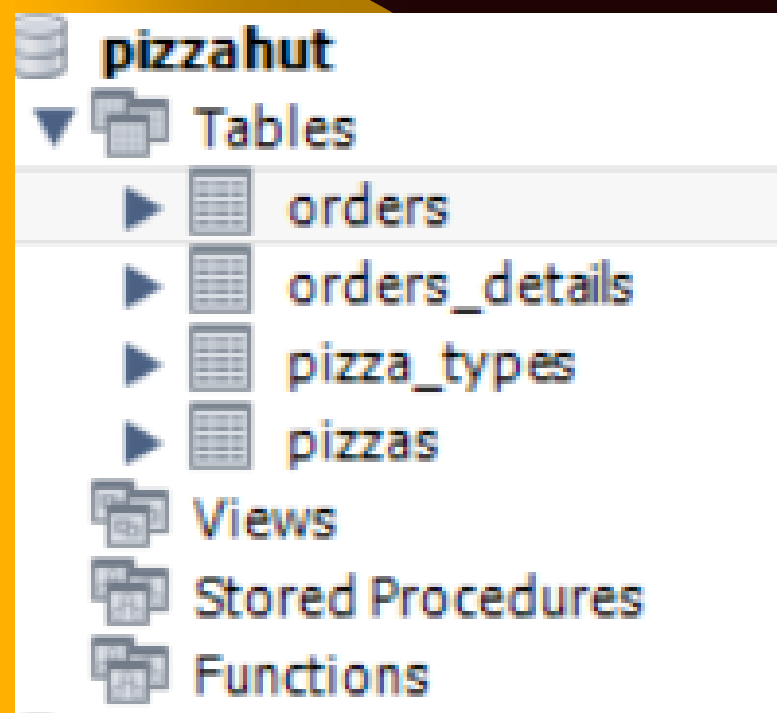
Describes the various types of pizzas available, including size and other specifications..

Key attributes: Pizza Type ID, Name, Category, Ingredients.

Pizzas:

Links the pizza types with specific sizes.

Key attributes: Pizza ID, Pizza Type ID, Size.





RETRIEVE THE TOTAL NUMBER OF ORDERS PLACED.

Contact

SQL QUERY:

```
SQL File 5*  SQL File 6* x  SQL File 7*  SQL File 8*  SQL File 9*  SQL File 10*  SQL File 11*
Limit to 1000 rows
1  -- Retrieve the total number of orders placed.
2 • select count(order_id) from orders as total_orders;
```

Result Grid		Filter
	count(order_id)	
▶	21350	

Pizzas:



CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES..

SQL QUERY:

```
1  -- Calculate the total revenue generated from pizza sales.
2  •  SELECT
3      ROUND (SUM(orders_details.quantity * pizzas.price), 2) AS total_sales
4  FROM
5      orders_details
6      JOIN
7      pizzas ON orders_details.pizza_id = pizzas.pizza_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	total_sales
▶	817860.05

Pizzas:



IDENTIFY THE HIGHEST-PRICED PIZZA. (NAME OF THE HIGHEST PRICE PIZZA)..

[About](#)[Contact](#)

SQL QUERY:

```
1  -- Identify the highest-priced pizza. (name of the highest price pizza)
2
3  • SELECT
4      pizza_types.name, pizzas.price
5  FROM
6      pizza_types
7      JOIN
8      pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
9  ORDER BY pizzas.price DESC
10 LIMIT 1;
11
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	name	price
▶	The Greek Pizza	35.95



IDENTIFY THE MOST COMMON PIZZA SIZE ORDERED ORDERS_DETAILS

[About](#)[Contact](#)

SQL QUERY:

```
2
3 • SELECT
4     pizzas.size,
5     COUNT(orders_details.order_details_id) AS order_count
6 FROM
7     pizzas
8     JOIN
9     orders_details ON pizzas.pizza_id = orders_details.pizza_id
10 GROUP BY pizzas.size
11 ORDER BY order_count DESC;
12
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	size	order_count
▶	L	18526
	M	15385
	S	14137
	XL	544
	XXL	28



LIST THE TOP 5 MOST ORDERED PIZZA TYPES ALONG WITH THEIR QUANTITIES.

[About](#)[Contact](#)

SQL QUERY:

```
2
3 • SELECT
4     pizza_types.name, SUM(orders_details.quantity) AS quantity
5 FROM
6     pizza_types
7     JOIN
8     pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
9     JOIN
10    orders_details ON orders_details.pizza_id = pizzas.pizza_id
11 GROUP BY pizza_types.name
12 ORDER BY quantity DESC
13 LIMIT 5;
14
```

Result Grid			Filter Rows:
	name	quantity	
▶	The Classic Deluxe Pizza	2453	
	The Barbecue Chicken Pizza	2432	
	The Hawaiian Pizza	2422	
	The Pepperoni Pizza	2418	
	The Thai Chicken Pizza	2371	



JOIN THE NECESSARY TABLES TO FIND THE TOTAL QUANTITY OF EACH PIZZA CATEGORY ORDERED.

[About](#)[Contact](#)

SQL QUERY:

```
SELECT
    pizza_types.category,
    SUM(orders_details.quantity) AS quantity
FROM
    pizza_types
    JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
    JOIN
    orders_details ON orders_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY quantity DESC;
```

Result Grid			Filter
	category	quantity	
▶	Classic	14888	
	Supreme	11987	
	Veggie	11649	
	Chicken	11050	



DETERMINE THE DISTRIBUTION OF ORDERS BY HOUR OF THE DAY

[About](#)[Contact](#)

SQL QUERY:

```
SELECT
    HOUR(order_time) AS hour, COUNT(order_id) AS order_count
FROM
    orders
GROUP BY HOUR(order_time);
```

Result Grid					
	hour	order_count			
	12	2520			
	13	2455			
	14	1472			
	15	1468			
	16	1920			
	17	2336			
	18	2399			
	19	2009			
	20	1642			
	21	1198			
	22	663			
	23	28			
	10	8			
	9	1			



JOIN RELEVANT TABLES TO FIND THE CATEGORY-WISE DISTRIBUTION OF PIZZAS.

[About](#)[Contact](#)

SQL QUERY:

```
SELECT
    category, COUNT(name)
FROM
    pizza_types
GROUP BY category;
```

Result Grid Filter Rows		
	category	COUNT(name)
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9



GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE NUMBER OF PIZZAS ORDERED PER DAY.

[About](#)[Contact](#)

SQL QUERY:

```
SELECT
    ROUND(AVG(quantity), 0) AS average_pizza_ordered_per_day
FROM
    (SELECT
        orders.order_date, SUM(orders_details.quantity) AS quantity
    FROM
        orders
    JOIN orders_details ON orders.order_id = orders_details.order_id
    GROUP BY orders.order_date) AS order_quantity;
```

Result Grid



Filter Rows:

	average_pizza_ordered_per_day
▶	138



DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE..

[About](#)[Contact](#)

SQL QUERY:

```
SELECT
    pizza_types.name,
    SUM(orders_details.quantity * pizzas.price) AS revenue
FROM
    pizza_types
    JOIN
    pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
    JOIN
    orders_details ON orders_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY revenue DESC
LIMIT 3;
```

Result Grid			Filter Rows:	
	name	revenue		
	The Barbecue Chicken Pizza	42768		
	The California Chicken Pizza	41409.5		
▶	The Thai Chicken Pizza	43434.25		



CALCULATE THE PERCENTAGE CONTRIBUTION OF EACH PIZZA TYPE TO TOTAL REVENUE..

[About](#)[Contact](#)

SQL QUERY:

```
SELECT
    pizza_types.category,
    (SUM(orders_details.quantity * pizzas.price) / (SELECT
        ROUND(SUM(orders_details.quantity * pizzas.price),
            2) AS total_sales
    FROM
        orders_details
        JOIN
        pizzas ON pizzas.pizza_id = orders_details.pizza_id)) * 100 AS revenue
FROM
    pizza_types
    JOIN
    pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
    JOIN
    orders_details ON orders_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY revenue DESC;
```

Result Grid		Filter Rows:
	category	revenue
	Classic	26.90596025566967
	Supreme	25.45631126009862
	Chicken	23.955137556847287
	Veggie	23.682590927384577



ANALYZE THE CUMULATIVE REVENUE GENERATED OVER TIME.

SQL QUERY:

```
select order_date,  
sum(revenue) over(order by order_date) as cum_revernue  
from  
(select orders.order_date,  
sum(orders_details.quantity * pizzas.price) as revenue  
from orders_details join pizzas  
On orders_details.pizza_id = pizzas.pizza_id  
join orders  
on orders.order_id = orders_details.order_id  
group by orders.order_date) as sales;
```

Result Grid		Filter Rows:
	order_date	cum_revernue
▶	2015-01-01	2713.8500000000004
	2015-01-02	5445.75
	2015-01-03	8108.15
	2015-01-04	9863.6
	2015-01-05	11929.55
	2015-01-06	14358.5
	2015-01-07	16560.7
	2015-01-08	19399.05
	2015-01-09	21526.4
	2015-01-10	23990.350000000002
	2015-01-11	25862.65
	2015-01-12	27781.7
	2015-01-13	29831.300000000003
	2015-01-14	32358.700000000004
	2015-01-15	34343.500000000001
	2015-01-16	36937.650000000001
	2015-01-17	39001.750000000001
	2015-01-18	40978.600000000006



DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE FOR EACH PIZZA CATEGORY.

[About](#)[Contact](#)

SQL QUERY:

```
select category, name, revenue
from
(select category, name, revenue,
rank() over(partition by category order by revenue desc) as rn
from
(select pizza_types.category, pizza_types.name,
sum((orders_details.quantity) * pizzas.price) as revenue
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
join orders_details
on orders_details.pizza_id = pizzas.pizza_id
group by pizza_types.category, pizza_types.name) as a) as b
where rn<=3;
```

Result Grid			
		Filter Rows:	Export:
	category	name	revenue
▶	Chicken	The Thai Chicken Pizza	43434.25
	Chicken	The Barbecue Chicken Pizza	42768
	Chicken	The California Chicken Pizza	41409.5
	Classic	The Classic Deluxe Pizza	38180.5
	Classic	The Hawaiian Pizza	32273.25
	Classic	The Pepperoni Pizza	30161.75
	Supreme	The Spicy Italian Pizza	34831.25
	Supreme	The Italian Supreme Pizza	33476.75
	Supreme	The Sicilian Pizza	30940.5
	Veggie	The Four Cheese Pizza	32265.700000000065
	Veggie	The Mexicana Pizza	26780.75
	Veggie	The Five Cheese Pizza	26066.5